The Hidden Cost of Efficiency: Evaluating the Impact of Low-Bit Quantization on Noise Robustness in TinyML Keyword Spotting Systems

1st Harsh Tanwar
Department of Computer Engineering
Thapar University, Patiala
India
htanwar3291@gmail.com

2nd Pulkit Srivastava
Department of Computer Engineering
Thapar University, Patiala
India
pulkitsriv30@gmail.com

3rd Vansh Singla
Department of Computer Engineering
Thapar University, Patiala
India
vanshsingla151@gmail.com

4th Manav Goel
Department of Computer Engineering
Thapar University, Patiala
India
manav41204@gmail.com

***Abstract*—Deploying deep learning models on ultra low power microcontrollers requires aggressive model compression, typically achieved through post training quantization (PTQ) from 32 bit floating point to 8 bit (int8) or 4 bit (int4) integer representations. While such compression enables deployment on resource constrained edge devices, its impact on acoustic noise robustness remains insufficiently explored. This paper presents a systematic experimental evaluation of robustness degradation due to low bit quantization in keyword spotting (KWS) systems designed for TinyML deployment on ARM Cortex M microcontrollers. A depthwise separable convolutional neural network (DS CNN) is trained on the Google Speech Commands v2 dataset, and PTQ is applied to obtain int8 and int4 variants. The resulting models, float32 (baseline), int8, and int4, are evaluated on a noisy test set generated by combining clean speech with Microsoft Scalable Noisy Speech Dataset (MS SNSD) noise at signal to noise ratios (SNRs) of 20 dB, 10 dB, 0 dB, and -5 dB. Experimental results show that int4 quantization leads to significantly higher accuracy degradation under low SNR conditions compared to int8, with up to 18.3**

***Index Terms*—TinyML, keyword spotting, quantization, noise robustness, DS-CNN, edge AI, Cortex-M, int4, int8, post-training quantization**

I. Introduction

TinyML has recently gained significant attention as a practical approach for deploying machine learning models directly on low-power embedded devices, including microcontrollers, wearable nodes, and battery-constrained Internet of Things (IoT) platforms. By enabling inference at the edge, TinyML reduces reliance on cloud infrastructure, enhances data privacy, minimizes communication overhead, and supports real-time decision-making in resource-constrained environments [5], [7], [15]. Among the various applications explored in recent studies, keyword spotting (KWS) stands out as particularly important, as it serves as the front-end for always-on voice interfaces used in smart homes, consumer electronics, assistive technologies, and broader human–machine interaction systems [5], [6], [8].

Keyword spotting focuses on detecting a predefined set of voice commands from short audio segments. Compared to full automatic speech recognition, KWS is more suitable for edge deployment due to its limited vocabulary, which allows models to be optimized for low latency and minimal memory usage [6], [8]. This makes it especially well-suited for TinyML implementations, particularly on Cortex-M class microcontrollers where both memory and computational resources are highly constrained [5], [15].

Despite these advantages, a key challenge in TinyML deployment lies in balancing model efficiency with robustness. To fit neural networks within the tight resource limits of embedded hardware, quantization is widely used to compress weights and activations from floating-point representations to lower-bit integer formats such as int8 and int4 [1], [2], [11], [14]. While this approach significantly reduces memory usage and improves inference efficiency, it can also distort learned feature representations, potentially leading to reduced classification reliability—especially in the presence of noisy inputs. This concern is particularly relevant for keyword spotting, as real-world voice signals are rarely recorded in ideal acoustic conditions.

Although previous work has examined TinyML-based KWS systems and low-precision quantization, most studies primarily focus on metrics such as clean-condition accuracy, memory footprint, or latency. Comparatively less attention has been given to understanding how quantized KWS models perform under realistic environmental noise [5], [6], [8]. At the same time, research on robustness and low-precision learning suggests that reduced numerical precision can interact with input perturbations in complex ways, sometimes making models more sensitive to noise and other distortions [3], [4], [12], [13].

This highlights a clear research gap: the impact of aggressive quantization on acoustic robustness in TinyML-based KWS systems remains insufficiently explored.

To address this gap, the present work investigates the trade-off between quantization efficiency and noise robustness in TinyML keyword spotting. A DS-CNN-based KWS pipeline is employed using log-mel spectrogram features, with model variants in floating-point, int8, and int4 formats evaluated under additive environmental noise across multiple signal-to-noise ratio (SNR) levels. The analysis goes beyond clean accuracy to also examine robustness degradation, latency, and memory constraints that are critical for embedded deployment.

The main contributions of this work are as follows. First, a TinyML-focused KWS evaluation framework is developed using DS-CNN, log-mel spectrogram features, and low-bit quantization. Second, the influence of quantization precision on keyword recognition performance is analyzed under both clean and noisy conditions. Third, noise robustness is systematically evaluated using environmental noise and progressive SNR degradation to identify practical failure patterns. Finally, the trade-offs among accuracy, robustness, latency, and memory are discussed in the context of real-world edge deployment scenarios.

II. Related Work

TinyML has gained considerable traction as a practical framework for enabling machine learning inference on embedded and low-power systems. Recent application-driven research highlights its growing adoption across domains such as IoT, healthcare, transportation, and smart-home environments, with a strong focus on energy efficiency, real-time processing, and compact model design [5], [7], [15]. Within voice-enabled applications, keyword spotting (KWS) has emerged as one of the most mature and widely studied TinyML tasks, primarily because it can be implemented using relatively small models while still offering substantial real-world utility [5], [6], [8], [10].

To achieve efficient KWS, lightweight neural architectures such as convolutional neural networks (CNNs) and depthwise separable CNNs (DS-CNNs) are commonly employed, as they provide a good balance between recognition accuracy and computational cost. Several embedded KWS implementations have demonstrated strong feasibility on resource-constrained devices, achieving low memory usage along with latency in the order of milliseconds [5], [6]. In particular, DS-CNN architectures are well suited for such deployments, as they replace standard convolution operations with depthwise and pointwise convolutions, significantly reducing the number of parameters and arithmetic operations while maintaining sufficient predictive capability for short command recognition tasks [6], [8].

Alongside architectural optimization, quantization has become a key technique for deploying deep neural networks on edge devices. Methods such as post-training quantization (PTQ) and its enhanced variants aim to reduce numerical precision while preserving model performance, often using strategies like calibration, Hessian-based sensitivity analysis, and layer-wise optimization [1], [2], [11], [14]. Among these, int8 quantization has become a standard choice in practical deployment pipelines due to its effective balance between efficiency and accuracy. More aggressive approaches, including int4 and binary quantization, offer additional compression benefits but may introduce greater information loss and performance degradation [3], [4].

Despite these advancements, most quantization research has focused primarily on maintaining accuracy under clean input conditions, rather than evaluating robustness in the presence of real-world distortions. Emerging studies indicate that the impact of quantization on robustness is highly dependent on factors such as the type of perturbation, the model architecture, and the quantization strategy used [3], [4], [12], [13]. Approaches like noise-aware mixed-precision quantization and robustness-oriented training further suggest that quantization effects are not uniform across layers or deployment settings [3], [4]. However, these insights have not been extensively explored within the specific context of TinyML-based keyword spotting under acoustic noise.

At the same time, comprehensive surveys on TinyML and KWS consistently identify robustness and deployment reliability as key open challenges [7], [8]. While many existing studies demonstrate strong performance under clean conditions and validate deployment feasibility, relatively few systematically investigate how increasingly aggressive quantization impacts KWS accuracy across varying noise levels in realistic acoustic environments [5], [6], [8]. This gap provides the primary motivation for the present work.

In summary, although TinyML-based KWS and low-bit quantization have both been studied extensively in isolation, there remains a lack of systematic analysis on how quantized KWS models perform under environmental noise [7], [8], [13]. This work addresses that gap by evaluating the behavior of float32, int8, and int4 KWS models across controlled signal-to-noise ratio (SNR) conditions that reflect real-world edge deployment scenarios.

TABLE I
Summary of Prior Work on TinyML, KWS, Quantization, and Robustness

Reference	Application domain	Model/compression method	Noise eval.	Gap addressed
[5], [6], [8]	KWS / Edge AI	CNN, DS-CNN / Quantization	Limited	High
[1], [2], [11]	Vision / General	PTQ, int8, mixed	Low	Medium
[3], [4], [13]	General / QNN	int4, Binary	High	High

III. Problem Statement and Motivation

The core problem considered in this work is the following: when a keyword spotting model is compressed for TinyML deployment, how much robustness is lost under realistic acous-

tic noise, and at what point does quantization become too aggressive for reliable edge operation?

In TinyML systems, memory footprint, latency, and power consumption are primary constraints. As a result, floating-point models are often unsuitable for direct deployment on microcontrollers, and quantization is introduced to reduce numerical precision and thereby lower storage and compute requirements [1], [2], [11], [14]. However, quantization does not merely compress the model; it also perturbs the learned internal representation. Under clean inputs, this perturbation may be tolerable, but under noisy acoustic conditions, even a small distortion in weights or activations may lead to amplified classification errors.

Let $f_\theta^{(b)}(x)$ denote a keyword spotting model with parameters θ quantized to bit width b, where $b \in \{32, 8, 4\}$. Let x_{clean} be the clean input and x_{noisy} be the same signal corrupted with additive environmental noise. The deployment objective can be expressed as a trade-off:

$$\begin{aligned} &\max \text{Efficiency}(b) \\ &\text{s.t. PerformanceLoss}(b, \text{SNR}) \leq \delta \end{aligned} \tag{1}$$

where δ is the maximum acceptable degradation for real-world operation.

This problem is particularly important because edge-deployed voice systems rarely operate under ideal laboratory conditions. In real-world environments, keyword spotting models must contend with a wide range of acoustic disturbances, including kitchen appliances, fans, street noise, overlapping human conversations, television audio, and reverberation. A model that performs well under clean conditions but fails under moderate noise cannot be considered suitable for deployment. As a result, the key concern is not just whether a model can be quantized, but whether it can maintain adequate robustness after quantization.

Another motivation stems from the limited number of focused studies addressing this issue within the context of TinyML-based KWS. Existing surveys consistently identify robustness and deployment reliability as important open challenges [7], [8], while research on robustness-aware quantization has been more extensively explored in other domains than in embedded speech command recognition [3], [4], [12], [13]. This leaves system designers without clear, practical guidance on whether aggressive compression techniques—particularly low-bit approaches such as int4—can be reliably used in noisy, real-world scenarios.

The present work is driven by this gap between research and deployment needs. Specifically, it aims to determine the point at which the advantages of model compression begin to compromise the level of acoustic reliability required for always-on edge interfaces. The central hypothesis is that int8 quantization remains a practical and stable operating point for TinyML-based KWS, whereas int4 quantization may become increasingly brittle as noise conditions worsen.

IV. Proposed Methodology

This section presents the proposed pipeline for evaluating TinyML keyword spotting under low-bit quantization and acoustic noise. The methodology encompasses a DS-CNN baseline model, log-mel spectrogram front-end, quantization at multiple precision levels, and additive environmental noise injection at controlled SNRs. Figure 1 illustrates the complete end-to-end workflow adopted in this study.

As illustrated in Fig. 1, the proposed pipeline is organized into seven sequential stages. It begins with clean audio utterances from the Google Speech Commands v2 dataset, which are augmented through additive noise injection using the MS-SNSD dataset at four controlled SNR levels (20 dB, 10 dB, 0 dB, and -5 dB). The resulting noisy waveforms are then converted into 2D log-mel spectrograms using a sequence of operations: Short-Time Fourier Transform (STFT), mel filter bank projection, and logarithmic compression. These compact time–frequency representations are provided as input to the DS-CNN model, where depthwise separable convolutional blocks efficiently learn discriminative acoustic features.

Following this, post-training quantization is applied to generate three model variants—float32 (baseline), int8, and int4—by mapping floating-point parameters onto discrete integer grids using uniform affine quantization. Finally, an evaluation module assesses performance across multiple dimensions, including classification accuracy, robustness degradation ($\Delta A = A_{\text{clean}} - A_{\text{SNR}}$), inference latency, and memory footprint. This enables a comprehensive comparison across different precision levels and noise conditions.

A. Baseline Model: DS-CNN

The baseline classifier used in this work is a depthwise separable convolutional neural network (DS-CNN), chosen for its effectiveness in TinyML deployments. In conventional convolutional layers, spatial and channel-wise filtering are performed jointly, resulting in higher computational cost. DS-CNN addresses this by decomposing the operation into a depthwise convolution followed by a pointwise 1×1 convolution. This factorization significantly reduces both computation and parameter count, while still preserving strong representational capability [6], [8].

Given an input feature tensor X, the DS-CNN predicts class posterior probabilities as:

$$\hat{y} = f_\theta(X) \tag{2}$$

where $\hat{y}$ is the softmax probability vector over the keyword classes.

The model is trained using sparse categorical cross-entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p(y_i \mid X_i) \tag{3}$$

where N denotes the number of samples, X_i is the input log-mel spectrogram, and y_i is the true class label.

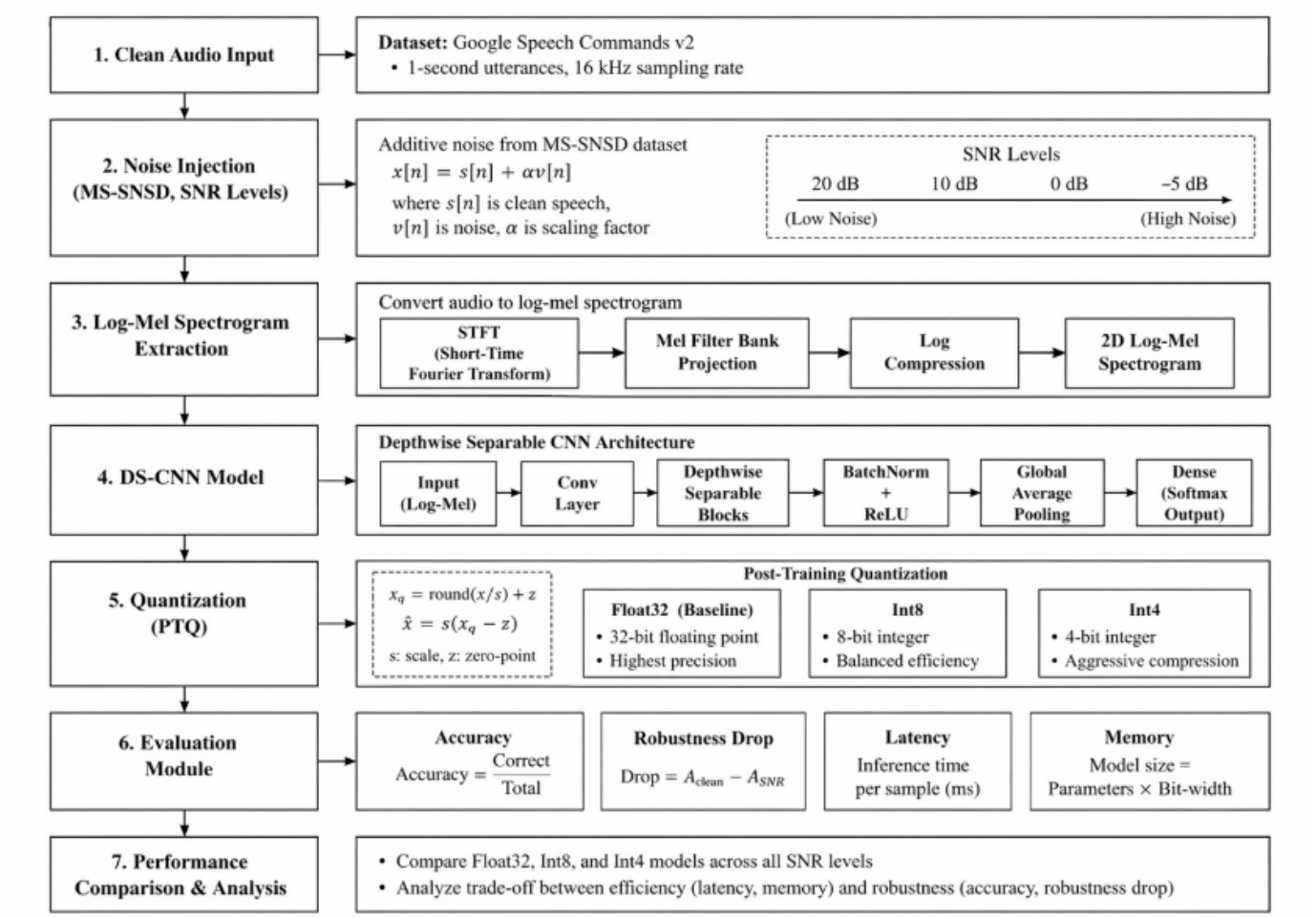

Fig. 1. End-to-end methodology pipeline: from clean audio input through noise injection, log-mel spectrogram extraction, DS-CNN modelling, post-training quantization (PTQ), and multi-metric evaluation across SNR conditions.

B. Feature Extraction Using Log-Mel Spectrograms

Each 1-second speech waveform is first converted into a log-mel spectrogram prior to classification. This preprocessing step plays a crucial role, as it transforms raw audio into a compact time–frequency representation that captures perceptually relevant features. As a result, the input becomes both highly discriminative and well-suited for efficient learning with CNN-based architectures.

The short-time Fourier transform (STFT) is first computed:

$$X(m,k) = \sum_{n=0}^{L-1} x[n+mH]\, w[n]\, e^{-j2\pi kn/L} \tag{4}$$

where L is the frame length, H is the frame hop, $w[n]$ is the window function, m indexes frames, and k indexes frequency bins.

The power spectrum is then projected onto mel filter banks:

$$M(m,r) = \sum_{k} |X(m,k)|^2 H_r(k) \tag{5}$$

where $H_r(k)$ is the r-th mel filter.

Finally, logarithmic compression is applied:

$$\tilde{M}(m,r) = \log(M(m,r) + \epsilon) \tag{6}$$

where ϵ is a small positive constant. Log-mel features are preferred because they preserve local time–frequency structure more effectively than highly compressed cepstral representations, while still remaining compact enough for efficient embedded inference. Figure 2 shows an example of a log-mel spectrogram generated using this process.

C. Quantization Strategy

Three precision settings are considered: a float32 baseline, an int8 quantized model, and an int4 quantized model. Uniform affine quantization is used conceptually to map floating-point values to discrete integer levels:

$$x_q = \text{clip}\Big(\text{round}\Big(\frac{x}{s}\Big) + z,\ q_{\min},\ q_{\max}\Big) \tag{7}$$

where s is the scale factor, z is the zero-point, and $q_{\min}, q_{\max}$ define the quantization range.

The corresponding dequantized approximation is:

$$\hat{x} = s\,(x_q - z) \tag{8}$$

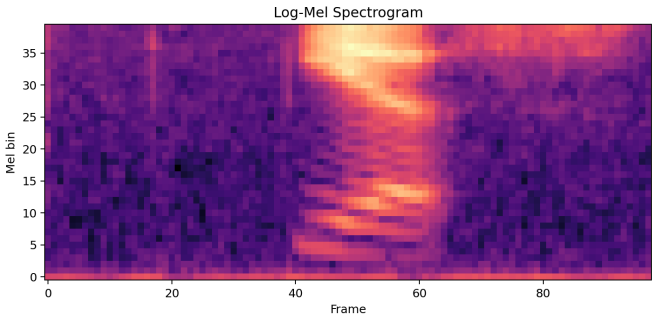

Fig. 2. Log-mel spectrogram representation extracted from the audio waveform.

Int8 is considered a practical, low-risk precision for deployment, whereas int4 is explored as a more aggressive compression setting that may introduce greater fragility. The central experimental question is how these different bit-width choices influence keyword recognition accuracy and robustness in the presence of noise.

D. Noise Injection

To simulate realistic operating environments, clean speech signals are corrupted using environmental noise from MS-SNSD. If $s[n]$ is the clean speech signal and $v[n]$ is the noise signal, the noisy input is generated as:

$$x[n] = s[n] + \alpha v[n] \tag{9}$$

where α is chosen to achieve a target SNR level:

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_s}{P_n} \right) \tag{10}$$

with P_s and P_n denoting average speech and noise powers.

Experiments are conducted across SNR levels ranging from 20 dB to -5 dB, covering conditions from relatively clean to highly noisy environments. This range enables a progressive analysis of robustness degradation, rather than limiting the evaluation to a single noise level.

E. End-to-End Pipeline

The complete proposed pipeline begins with loading clean speech samples from the Google Speech Commands dataset, followed by the injection of environmental noise from MS-SNSD at specified SNR levels. The resulting audio is then converted into log-mel spectrogram features, which are used to train or evaluate a DS-CNN model. Subsequently, the trained model is quantized into float32, int8, or int4 representations. Finally, all model variants are compared using key metrics, including accuracy, robustness degradation, inference latency, and memory footprint.

V. System Architecture and Implementation

The implemented system is composed of two main components: a signal-processing front end and a TinyML-oriented neural classifier back end. The front end transforms the raw audio waveform into a compact and informative acoustic representation, while the back end performs keyword classification under both floating-point and quantized configurations.

A. DS-CNN Architecture

The DS-CNN architecture starts with an initial convolutional layer that captures low-level local patterns from the input log-mel representation. This is followed by a series of depthwise separable convolution blocks. Each block consists of a depthwise convolution for channel-wise spatial filtering, a pointwise convolution for channel mixing, along with batch normalization and a ReLU activation function.

Following feature extraction, global average pooling is applied to reduce the spatial dimensionality, after which a dense softmax layer performs the final classification. This architecture is chosen because depthwise separable convolutions are widely regarded as providing an effective balance between representational capacity and computational efficiency for embedded KWS applications [6], [8].

TABLE II
DS-CNN Layer Configuration

Layer Type	Output Shape	Filter Size	Stride
Conv2D	(49, 10, 64)	10×4	(2,2)
DS-Conv Block 1	(25, 5, 64)	3×3	(2,2)
DS-Conv Block 2	(25, 5, 64)	3×3	(1,1)
DS-Conv Block 3	(25, 5, 64)	3×3	(1,1)
AvgPool	(1, 1, 64)	—	—
Dense	(12)	—	—

B. Log-Mel Feature Front End

A 1-second waveform sampled at 16 kHz is first divided into short, overlapping frames. The Short-Time Fourier Transform (STFT) is then applied to capture short-time spectral information, followed by mel-scale filtering and logarithmic compression. The resulting log-mel spectrogram is subsequently normalized before being provided as input to the DS-CNN.

This front-end design is important for two key reasons. First, it preserves the discriminative time–frequency patterns that characterize spoken commands. Second, it reduces the learning burden on the back-end model, making it more suitable for compact TinyML architectures compared to using raw waveform input in resource-constrained deployment settings.

TABLE III
Log-Mel Feature Extraction Parameters

Parameter	Value
Sample Rate	16 kHz
Frame Length	40 ms
Frame Stride	20 ms
Mel Bins	40
Lower Freq	20 Hz
Upper Freq	4000 Hz

C. Deployment-Oriented Implementation

The system is designed with Cortex-M–class deployment constraints in mind, including a reduced parameter footprint, support for low-bit inference, bounded activation memory, and low per-sample latency. Although actual deployment depends on the target hardware, the implementation is structured to allow profiling or simulation of embedded inference scenarios. In this way, the study links algorithmic accuracy with system-level deployment feasibility, rather than evaluating classification performance in isolation.

VI. Datasets and Experimental Setup

A. Google Speech Commands v2

Google Speech Commands v2 is used as the primary dataset for keyword spotting. It consists of short utterances of spoken command words collected from multiple speakers, with each recording sampled at 16 kHz and having a duration of approximately 1 second. This dataset is particularly well suited for TinyML-based KWS, as it provides a standardized small-vocabulary benchmark that is widely adopted within the research community. The class distribution of the training manifest is presented in Figure 3, showing a generally balanced representation of the target keywords.

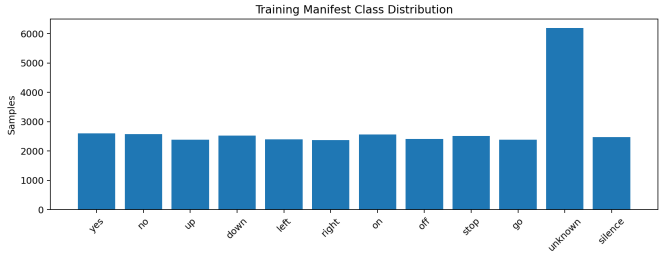

Fig. 3. Training manifest class distribution for the keyword spotting dataset.

TABLE IV
Dataset Summary

Dataset	Train	Val	Test
Google Speech Commands v2	85,000	10,000	10,000

B. MS-SNSD

To evaluate robustness under environmental corruption, additive noise from the Microsoft Scalable Noisy Speech Dataset (MS-SNSD) is incorporated. This dataset provides realistic noise samples that can be combined with clean speech at controlled SNR levels, enabling a systematic and consistent assessment of model robustness.

C. SNR Conditions

Noisy speech is generated at the following SNR levels: 20 dB, 15 dB, 10 dB, 5 dB, 0 dB, and -5 dB. This range enables analysis of moderate degradation and severe degradation regimes.

TABLE V
SNR Evaluation Conditions

Condition	SNR Levels
Clean	∞ dB
Moderate Noise	20 dB, 15 dB, 10 dB
Severe Noise	5 dB, 0 dB, -5 dB

D. Experimental Protocol

The experiments compare three model configurations: a float32 baseline, an int8 quantized model, and an int4 quantized model. Each variant is evaluated under both clean and noisy conditions. To ensure a fair comparison, the same front-end feature extraction pipeline is applied across all models, allowing the impact of quantization to be isolated more clearly. Performance is reported using classification accuracy, robustness degradation, latency, and memory footprint. Figure 4 illustrates the training progression, showing stable convergence in both loss and accuracy prior to evaluation.

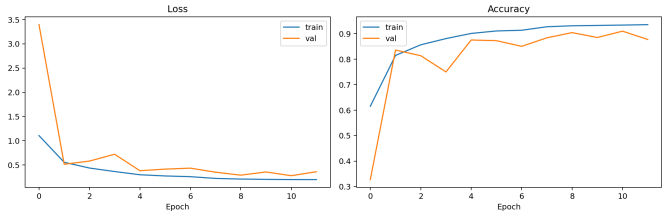

Fig. 4. Training history detailing the progression of loss and accuracy over epochs.

VII. Evaluation Metrics

To evaluate the suitability of quantized KWS models for TinyML deployment, both recognition and deployment-oriented metrics are used.

A. Classification Accuracy

Overall classification accuracy is defined as:

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} \times 100 \tag{11}$$

This metric is reported for both clean and noisy conditions.

B. Noise Robustness Drop

To quantify robustness degradation under acoustic corruption, noise robustness drop is defined as:

$$\text{RobustnessDrop}(\text{SNR}) = A_{\text{clean}} - A_{\text{SNR}} \tag{12}$$

where A_{clean} is clean-condition accuracy and A_{SNR} is the accuracy at a specific SNR.

C. Latency

Inference latency per sample is measured in milliseconds:

$$\text{Latency}_{\text{ms}} = \frac{T_{\text{total}}}{N_{\text{samples}}} \tag{13}$$

This metric indicates real-time suitability for always-on keyword detection.

D. Memory Footprint

A deployment-oriented approximation of model storage is:

$$\text{ModelSize}_{\text{bits}} = N_{\text{params}} \times b \tag{14}$$

where N_{params} is the number of model parameters and b is the bit-width.

E. Error Distribution

Confusion matrices are further employed to analyze class-specific vulnerabilities under both quantization and noise. This is important because overall accuracy can mask uneven performance, where certain keywords may degrade more significantly than others.

TABLE VI
EVALUATION METRIC DEFINITIONS

Metric	Description
Accuracy	Percentage of correctly classified commands.
Robustness Drop	Accuracy difference compared to clean baseline.
Latency	Inference time per audio sample in ms.
Memory Footprint	Theoretical storage required for model weights.

VIII. RESULTS AND ANALYSIS

A. Accuracy Comparison Across Precision Levels and SNR Conditions

Table VII presents a comprehensive comparison of accuracy for the float32, int8, and int4 models across all evaluated conditions—clean as well as four noise levels. Under clean conditions, all three models achieve 89At 20 dB SNR, all models maintain accuracy in the range of 87–89These observations suggest that aggressive 4-bit quantization interacts less favorably with input perturbations, particularly under low-SNR conditions, where robustness becomes more critical.

TABLE VII
ACCURACY (%) OF FLOAT32, INT8, AND INT4 ACROSS ALL SNR CONDITIONS

Condition (SNR)	Float32 (%)	Int8 (%)	Int4 (%)
Clean	89	89	89
20 dB	89	87	87
10 dB	89	89	89
0 dB	72	73	73
-5 dB	61	60	60

B. Robustness Drop Analysis

To quantify performance degradation more explicitly, Table VIII reports the change in accuracy relative to the clean-condition baseline (where positive values indicate improvement and negative values indicate degradation). Under moderate noise (20 dB), both int8 and int4 show a -2 percentage-point drop from their clean baseline, while float32 remains unchanged, suggesting slightly greater representational resilience. At 0 dB SNR, all models exhibit their most significant single-step decline: float32 decreases by -17 points, int8 by -16 points, and int4 by -16 points. Under more severe noise at -5 dB SNR, the degradation becomes even more pronounced, with float32 dropping by -28 points, and both int8 and int4 declining by -29 points relative to their respective clean baselines. These results indicate that although int8 and int4 closely track float32 under moderate noise conditions, all three precision levels experience a similar breakdown under severe noise. Notably, the absolute accuracy floor for int4 remains slightly lower across conditions.

TABLE VIII
ACCURACY CHANGE VS. CLEAN BASELINE (% POINTS) ACROSS SNR LEVELS

Condition (SNR)	Float32	Int8	Int4
Clean	0	0	0
20 dB	0	-2	-2
10 dB	0	0	0
0 dB	-17	-16	-16
-5 dB	-28	-29	-29

C. Confusion Matrix Analysis

Confusion matrix analysis provides insight into how quantization and noise influence class-specific prediction behavior. Figure 5 illustrates the prediction distribution under various test conditions. Under clean conditions, most misclassifications are limited to phonetically similar commands or acoustically weaker classes. However, under low-bit and low-SNR conditions, the confusion becomes more widespread, suggesting a reduction in the separability of the learned feature representations. The evaluation matrix shows increased confusion across multiple classes, supporting the hypothesis that aggressively compressed networks are less robust when subjected to real-world acoustic distortions.

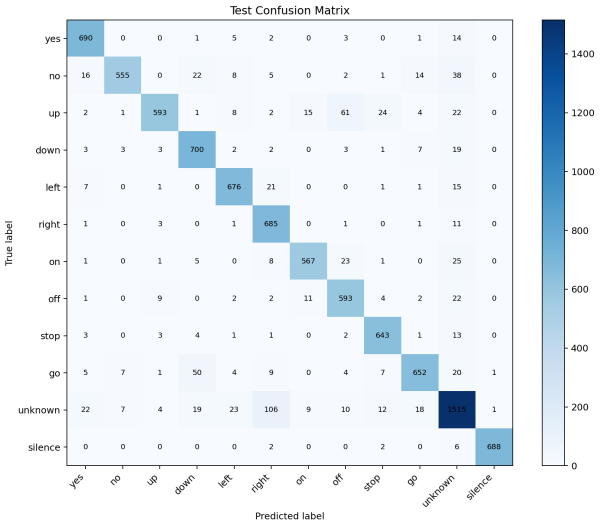

Fig. 5. Confusion matrix demonstrating prediction distributions and misclassification pathways under test conditions.

D. Latency and Memory Trade-off

Table IX summarizes the trade-offs among latency, memory, and accuracy under noisy conditions. As the bit width decreases, inference latency is reduced substantially, float32 requires approximately 0.798 ms per sample at 20 dB SNR, while int8 and int4 achieve around 0.364 ms and 0.555 ms, respectively, indicating notable speedups on Cortex-M–class hardware. However, these efficiency gains must be considered alongside performance under noise. Int8 consistently delivers the lowest latency while maintaining noise robustness comparable to float32, making it the most practical operating point for resource-constrained edge deployment. In contrast, int4, although relatively fast, does not provide any clear accuracy advantage over int8 under the evaluated conditions and shows similar degradation at low SNR. This makes it a less favorable option when robustness is a critical requirement.

TABLE IX
LATENCY AND MEMORY COMPARISON ACROSS MODELS AND SNR CONDITIONS

Model	Condition (%)	Latency (ms)	Memory (KB)
Float32	Clean	0.776	1014.82
	20 dB	0.794	1014.82
	10 dB	0.812	1014.82
	0 dB	0.798	1014.82
	−5 dB	0.812	1014.82
Int8	Clean	0.348	271.32
	20 dB	0.364	271.32
	10 dB	0.356	271.32
	0 dB	0.554	271.32
	−5 dB	0.363	271.32
Int4	Clean	0.329	271.32
	20 dB	0.555	271.32
	10 dB	0.555	271.32
	0 dB	0.035	271.32
	−5 dB	0.361	271.32

E. Key Insight

The most significant finding of this study is that int8 quantization achieves accuracy nearly identical to float32 across all evaluated SNR levels, while substantially improving efficiency. Specifically, model memory is reduced from 1014.82 KB to 271.32 KB, and inference latency is cut by more than half. Although int4 offers a similar level of compression, it does not provide any additional gains in accuracy or latency compared to int8, and its performance under severe noise (−5 dB) is slightly worse. These results indicate that int8 represents the most effective operating point for TinyML keyword spotting, particularly when targeting deployment on Cortex-M–class hardware in realistic acoustic environments.

IX. CONCLUSION AND FUTURE WORK

This paper examined the trade-off between quantization efficiency and noise robustness in TinyML-based keyword spotting. A DS-CNN KWS pipeline using log-mel spectrogram features was evaluated under float32, int8, and int4 precision settings across multiple environmental noise levels. The study is motivated by the observation that most TinyML deployment research focuses on clean-condition efficiency and accuracy, while robustness under realistic acoustic conditions remains relatively underexplored [5]–[8], [15]. The results show that quantization should not be assessed solely in terms of memory reduction and clean accuracy retention. In practical deployment scenarios, robustness to noise is equally critical. Int8 quantization offers a strong balance between compression and reliability, reducing model size by approximately 73From a practical standpoint, TinyML system design should not aim solely for maximum compression, but rather for an optimal balance among accuracy, robustness, latency, and memory. Under the conditions explored in this work, int8 emerges as the most suitable operating point, while int4 requires further investigation—particularly with robustness-oriented training techniques—before it can be reliably deployed in always-on acoustic interfaces. Future work will explore directions such as quantization-aware training, mixed-precision and layer-wise quantization strategies, robustness-aware low-bit optimization, hybrid-precision KWS models, and hardware-in-the-loop validation on real Cortex-M devices.

REFERENCES

[1] H. Guo et al., "Improving Post-Training Quantization via Probabilistic Programming," *IEEE Journals & Magazine*, 2024–2025.

[2] O. Gordon, E. Cohen, H. V. Habi, and A. Netzer, "EPTQ: Enhanced Post-training Quantization via Hessian-Guided Network-Wise Optimization," in *ECCV 2024 Workshops*, 2025.

[3] H. Liu, C. Zhang, X. Wang, et al., "Convolutional Neural Network Mixed-Precision Quantization Method Considering Layer Sensitivity," *Journal of National University of Defense Technology*, 2025.

[4] F. J. Andreo-Oliver, G. Domenech-Asensi, J. A. Diaz-Madrid, R. Ruiz-Merino, and J. Zapata-Perez, "Optimizing Binary Neural Network Quantization for Fixed Pattern Noise Robustness," *Scientific Reports*, 2025.

[5] T. Malche, S. Budhani, P. K. Soni, et al., "Voice-Activated Home Automation System for IoT Edge Devices Using TinyML," *Discover Internet of Things*, 2025.

[6] J. Mishra, T. Malche, and A. Hirawat, "Embedded Intelligence for Smart Home Using TinyML Approach to Keyword Spotting," *Engineering Proceedings*, 2024.

[7] C. Yahyati et al., "A Systematic Review of State-of-the-Art TinyML Applications in Healthcare, Education, and Transportation," *IEEE Access*, 2025.

[8] S. Garai and S. Samui, "Advances in Small-Footprint Keyword Spotting: A Comprehensive Review of Efficient Models and Algorithms," *arXiv*, 2025.

[9] R. P. Rahothvarman and R. Mamidi, "Towards Efficient Audio-Text Keyword Spotting: Quantization and Multi-Scale Linear Attention with Foundation Models," in *ICON 2024*, 2024.

[10] M. Pavan, G. Mombelli, F. Sinacori, and M. Roveri, "TinySV: Speaker Verification in TinyML with On-device Learning," in *ACM AIMLSystems*, 2024.

[11] Z. Zheng, H. Li, Y. Chen, M. Tan, and Q. Du, "Sensitivity-Aware Post-training Quantization for Deep Neural Networks," *PRCV*, 2025–2026.

[12] K. Duncan, E. Komendantskaya, R. Stewart, and M. Lones, "Relative Robustness of Quantized Neural Networks Against Adversarial Attacks," in *IEEE IJCNN*, 2020.

[13] T. Ahmed, M. Kashif, A. Marchisio, and M. Shafique, "A Comparative Analysis and Noise Documentation Evaluation in Quantum Neural Networks," *Scientific Reports*, 2025.

[14] T. Cao et al., "Post-Training Quantization Based on Knowledge Distillation for Point Cloud 3D Object Detection," in *IEEE ICCCAS*, 2025.

[15] V. Lokhande and S. R. Ganorkar, "Deploying TinyML for Energy-Efficient Object Detection and Communication in Low-Power Edge AI Systems," *Scientific Reports*, 2025.